

Assignment 01: Hosting and Load Balancing a Web Application Using Ubuntu Virtual Machines

Milestone 01: Develop a Sample Web Application

Develop a quadratic equation solver web application using HTML, jQuery, Bootstrap, and your choice of server-side technology such as PHP, Rails, Django, Spring MVC, etc. Instead of accepting the values of a , b , and c from the user, implement a server-side function that randomly generates three numbers a , b , and c each time it is called, ensuring that $a \neq 0$. The function should use these values to form and solve the quadratic equation $ax^2 + bx + c = 0$ and return the generated values of a , b , and c , along with the computed solution(s). The application should handle cases where the equation has two real solutions, one real solution, or no real solutions. In addition, the function must return the IP address and/or hostname of the server hosting the application. This information will be used later when the application is replicated across multiple virtual machines (VMs) to identify which VM processed each request. The web interface should display the generated equation, its coefficients, the calculated solution(s), and the server's IP address and/or hostname in a clear and user-friendly manner.

The deliverable of this milestone is to provide a snapshot of your user interface with an output to some sample input and source code of your server-side implementation.

Milestone 02: Host your Web Application on Ubuntu Virtual Machines

In this part of the, you will provision your infrastructure using full virtual machines. Create two Ubuntu Server VMs (Ubuntu 22.04 LTS or later) using a hypervisor of your choice (VirtualBox, VMware Workstation/Player, or KVM/QEMU). Each VM should be configured with a bridged or host-only network adapter so that it is reachable from your host operating system.

On each VM, install the software stack required to run your web application (e.g., a web/application server, runtime, and any dependencies) and deploy your application from Milestone 01 onto both VMs independently. You must be able to access and use your web application from your host operating system's browser by visiting the IP address of each VM directly.

You should document, as part of your report, how each VM was provisioned: the base image used, allocated resources (vCPU, RAM, disk), the network configuration, and the steps taken to install and start your application.

A snapshot of the browser showing the application running when accessed from each VM's IP address, a snapshot/output showing both VMs running simultaneously (e.g., output of "VBoxManage list runningvms", "virsh list", or your hypervisor's VM manager), and the network/provisioning configuration used for each VM.

Milestone 03: Deploy Nginx as a Proxy Server in a Virtual Machine

Now, provision a third Ubuntu VM dedicated to acting as a reverse proxy and load balancer. On this VM, install Nginx directly on the operating system (i.e., via apt, not inside a container) and configure it as a proxy server that load balances incoming traffic across the two application VMs you deployed in Milestone 02.

Configure the Nginx "upstream" block to list the private/bridged IP addresses of both application VMs, and verify that repeated requests to the proxy VM are distributed across both backend VMs (you can use the IP/hostname returned by your server-side function from Milestone 01 to confirm this).

A snapshot of the browser showing the application being accessed through the Nginx proxy VM's IP address, the Nginx configuration file used for proxying and load balancing, and a snapshot/output confirming all three VMs (two application VMs + one proxy VM) are running.

Milestone 04: Synthetic Workload Generation

In this part of the assignment, you will use `httperf` to generate synthetic workload for your application. You have to generate traffic directed at your Nginx VM, which will automatically load balance the traffic across the two backend application VMs. The instructor will email you the basic `httperf` script and some tips. You need to store the output of `httperf` and profile the CPU utilization of your proxy VM (and/or host machine) during the experiment. Remember, you must build a small utility to profile the CPU of your machine during the load test.

In this part of the assignment, you will be using `httperf` to generate the synthetic workload for your application. You have to generate the traffic to your Nginx container, and it will automatically load balance the traffic. The instructor will email you the basic script of `httperf` and some tips. You need to store the output of `httperf` and needs to profile the CPU of your system. Remember! You have build a small utility to profile the CPU of your machine.

The deliverable of this milestone is to provide two graphs: the first graph shows the average response time of the application (you will get it from `httperf`'s output) and the second graph will show CPU utilization during this experiment.

Submission Guidelines

You need to prepare a small report in PDF format containing deliverables for all the milestones before **September 28, 2026, 11:59 pm via Google Classroom**.

References

- [1] `httperf`: A Tool for Measuring Web Server Performance. <http://www.labs.hpe.com/research/linux/httperf/docs.php>.